

TP Note JS elo ranker

Shanka Clermont

Base de données

Prisma 6.11 (basé sur les connaissances de la SAE)

Installation du projet

```
git clone https://github.com/Emir-de-la-moquette/TPnote-ranker-elo.git

cd ./TPnote-ranker-elo

pnpm install

# Migration joueur / match si aucune migration n'existe
pnpm prisma migrate dev --name init-player-match

# Sinon, utiliser la migration joueur / match déjà réalisée (meilleure option)
pnpm prisma migrate deploy

# Si les migrations sont déjà appliquées mais que le client n'est pas à jour :

pnpm prisma generate

## Lancer le projet

# build l'ui
pnpm run libs:ui:build

# Lancer le client
pnpm run apps:client:start # lance sur localhost:3001

# Puis sur un second terminal

# Lancer le serveur
pnpm run apps:server:start
```

Objectifs

-  Créer un serveur HTTP en utilisant NestJS

- ☒ Créer un service de type Singleton pour stocker des données du classement en cache
- ☒ Créer un service pour écrire et lire les données des joueurs en base de données (prisma)
- ☒ Ajouter la logique métier pour mettre à jour le classement des joueurs
- ☒ Ajouter un EventEmitter pour émettre des notifications en temps réel
- ☒ Implémenter l'API décrite par le Swagger fourni
- ☒ Tester l'application avec des tests unitaires et des tests d'intégration

Fonctionnalités

- ☒ Un joueur peut être créé avec un elo initial qui est la moyenne de tous les elos des joueurs existants, et une valeur par défaut au cas où.
- ☒ Un match peut être créé entre deux joueurs.
- ☒ Visualiser les joueurs et leurs elos.
- ☒ Un historique des matchs est stocké.
- ☒ Les elos des joueurs sont mis à jour en temps réel.
- ☒ Les joueurs sont stockés en mémoire cache et en base de données.
- ☐ match aléatoire entre deux joueurs toutes les XX secondes.
- ☐ Simulation de match.

Tests

lancer les tests

```
# À la racine du projet
clear ; pnpm apps:server:test

lancer le coverage

# À la racine du projet
clear ; pnpm apps:server:test:coverage
```

Résultats

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	89.63	84.61	96.66	89.13	
src	44.82	0	75	39.13	
app.controller.ts	100	100	100	100	
app.module.ts	0	100	100	0	1-20
app.service.ts	100	100	100	100	
main.ts	0	0	0	0	1-10
src/match	97.72	85.71	100	97.36	
match.controller.ts	100	100	100	100	
match.module.ts	100	100	100	100	
match.service.ts	96	85.71	100	95.65	38
src/players	100	100	100	100	
players.controller.ts	100	100	100	100	
players.module.ts	100	100	100	100	
players.service.ts	100	100	100	100	
src/rankings	100	100	100	100	
rankings.controller.ts	100	100	100	100	
rankings.module.ts	100	100	100	100	
rankings.service.ts	100	100	100	100	
Test Suites: 10 passed, 10 total					
Tests: 35 passed, 35 total					